CARDINAL ROUTER

4×4 Mesh Network-on-Chip · Phase 1 Report

HOP-Based | XY-Standard | Torus Routing

Team Members	Course Information
Om Bhatt · Melvin Houston	EE577B · Prof. Moe Tabar · Spring 2026

328/328
Test Cases Passed
All gather phases

259.7 MHz
Max Frequency
XYSTD variant

24.6%
Latency Reduction
Torus vs Mesh

~30,000 µm²
Cell Area
Per router (all 3)

University of Southern California · Viterbi School of Engineering

1. Project Overview

This report documents the design, implementation, verification, and synthesis of a Cardinal Router-based 4×4 Mesh Network-on-Chip (NoC) for EE577B. The project was completed in three parts, each adding a layer of complexity and optimization over the prior variant.

1.1 Design Objectives

- Implement a fully synthesizable RTL Cardinal Router with virtual cut-through routing
- Connect 16 routers in a 4×4 bidirectional mesh using XY dimension-order routing
- Implement a Cardinal NIC for processor-to-NoC interfacing
- Evaluate three routing variants: HOP-based, XY-Standard (XYSTD), and Torus
- Verify functional correctness with 328 total test assertions across all scenarios

1.2 Design Variants Summary

Variant	Routing Mechanism	Header Modification	Topology
HOP-based	Unary hop counter + direction bits	Yes — decrement at each router	4×4 Mesh (open edges)
XY-Standard	Absolute dst_x/dst_y comparison	None — static header	4×4 Mesh (open edges)
Torus	Modular 2-bit subtraction (dst−src) mod 4	None — static header	4×4 Torus (wrap links)

2. Design Architecture & Hierarchy

2.1 Module Hierarchy

The design follows a strict hierarchical structure with each module encapsulating a single responsibility. The hierarchy for the HOP-based variant is shown below; XYSTD and Torus variants follow the same structure with renamed modules.

Module	File	Responsibility
gold_mesh	gold_mesh.v	4×4 NoC top-level, connects 16 routers
gold_router	gold_router.v	Single router: instantiates all submodules
input_ctrl (×5)	input_ctrl.v	Per-port input: VC buffers, routing decode, handshake
output_ctrl (×5)	output_ctrl.v	Per-port output: arbitration, forwarding, Mealy so
vc_buffer (×10)	vc_buffer.v	64-bit registered VC storage (even/odd per port)
rr_arbiter (×5)	rr_arbiter.v	Round-robin arbiter for output port contention
cardinal_nic	cardinal_nic.v	NIC: processor ↔ router packet interface

2.2 Packet Header Format

Two distinct 64-bit packet formats are used depending on the routing variant:

HOP-Based Header (64-bit)

[63]	[62]	[61]	[60:56]	[55:52]	[51:48]	[47:0]
VC polarity	dirX (0=E)	dirY (0=N)	Reserved	hopX (unary)	hopY (unary)	src_x, src_y, payload

XYSTD / Torus Header (64-bit)

[63]	[62:56]	[55:48]	[47:40]	[39:32]	[31:24]	[23:0]
VC polarity	Reserved (7b)	dst_x (8b)	dst_y (8b)	src_x (8b)	src_y (8b)	Payload (24b)

2.3 Key Design Decisions

Virtual Channel Double-Buffering

Each input port maintains two VC buffers (even and odd). The polarity register toggles every clock cycle, creating alternating write/service cycles. This prevents read-write collisions and enables pipeline throughput without deadlock.

- Even cycle (polarity=0): external port writes to odd buffer; even buffer is serviced
- Odd cycle (polarity=1): external port writes to even buffer; odd buffer is serviced
- Injection protocol: assert pesi when polarity=1 AND peri=1 at negedge clk

Mealy vs. Moore FSM Selection

Signal	FSM Type	Rationale
ri (Ready-In)	Moore — output = f(state only)	Buffer full/empty is internal state only; no combinational path from inputs needed
so (Send-Out)	Mealy — output = f(state, ro)	Must respond to ro combinationally; waiting a clock edge would add +1 cycle per hop

3. Test Strategy & Verification

The testbench (tb_gold_mesh.v) implements 328 total assertions organized into four categories, systematically covering every routing scenario, handshake condition, arbitration case, and the full mesh gather operation.

3.1 Test Categories Overview

Category	Test Cases	Assertions	Coverage
Reset & Basic Routing	TC1–TC9	~88	All 4 directions, 1/2/3-hop, full diagonal 6-hop
Handshaking	TC10–TC11	~12	Non-blocking pipeline, blocking stall/flush
Contention & Arbitration	TC12–TC13	~12	4-way contention, 2-round fairness verification
Full Mesh Gather	TC14	240	16 phases × 15 senders, scoreboard validation
TOTAL	**14 TCs**	**328**	**100% PASS — all assertions verified**

3.2 Basic Routing Tests (TC1–TC9)

TC	Route	Path	Hops	What It Verifies
TC1	Reset verification	All nodes	—	peso=0, peri=1, polarity=0→1→0
TC2	(0,0)→(1,0)	East 1 hop	1	East port routing, hopX 4'h1→0
TC3	(0,0)→(0,1)	North 1 hop	1	North port, hopY 4'h1→0
TC4	(3,0)→(2,0)	West 1 hop	1	West port, dirX=1 decoded correctly
TC5	(0,3)→(0,2)	South 1 hop	1	South port, dirY=1 decoded correctly
TC6	(0,0)→(3,0)	3-hop East	3	Multi-hop: hopX 4'h7→4'h3→4'h1→4'h0
TC7	(0,0)→(0,3)	3-hop North	3	Multi-hop Y dimension, hopY decrement chain
TC8	(0,0)→(3,3)	Diagonal NE	6	XY-order: 3 hops East THEN 3 hops North
TC9	(3,3)→(0,0)	Diagonal SW	6	West+South multi-hop, direction bits both set

3.3 Handshaking Tests (TC10–TC11)

These tests verify the Mealy/Moore handshake protocol under both ideal and adversarial conditions.

TC10 — Non-Blocking (Back-to-Back Injection)

- Two packets injected consecutively from node00 to node10 with pero=1 held throughout
- Expected: Packet 1 arrives cycle T+4; Packet 2 arrives cycle T+6 (2-cycle pipeline separation)
- Verifies: VC double-buffering allows pipelined transmission without stalls
- Pass condition: both packets arrive with correct source ID, zero drops

TC11 — Blocking (Stall and Flush)

- Packet injected; pero at destination deasserted before packet arrives
- Expected: packet held in output VC buffer; peri at upstream router drops to 0; pipeline backs up
- pero reasserted; packet flushes on next polarity-correct cycle
- Verifies: Mealy so responds combinationally to ro; no extra cycle wasted; no packet loss

3.4 Contention & Arbitration Tests (TC12–TC13)

TC12 — 4-Way Contention

- Nodes (0,2), (3,2), (2,0), (2,3) all inject simultaneously toward node(2,2)
- All 4 packets arrive at node22's PE output controller — 4 simultaneous requests
- Expected: rr_arbiter grants one per cycle; all 4 packets delivered in 4 consecutive cycles
- Verifies: no double-grant, no packet loss, correct FIFO ordering through arbitration

TC13 — Round-Robin Fairness

- Same 4-node contention repeated for 2 full rounds (8 total packets)
- Priority pointer must advance after each contention event
- Verifies: the winner of round 1 is NOT the same as round 2 — pointer rotation confirmed
- Critical for anti-starvation; a broken arbiter passes TC12 but fails TC13

3.5 Full Mesh Gather (TC14)

The gather test is the most comprehensive verification: 16 sequential phases, each with 15 simultaneous senders, producing 240 total packet delivery assertions.

- Phase k: all 15 nodes except node k inject a packet destined for node k simultaneously
- Scoreboard at each node: 15-bit vector, one bit per possible source
- Phase completes only when all 15 bits are set (all sources received)
- Phase k+1 begins only after phase k is confirmed complete (sequential phasing)
- Each packet logged to gather_phasek.res: Phase=, Time=, Destination=, Source=, Packet Value=
- Start and end times of each phase logged to start_end_time.out

Stress factors in TC14:

- Simultaneous traffic on all inter-router links
- Multi-path congestion: corner nodes must traverse 6 hops while near nodes arrive in 4 cycles
- Implicit VC multiplexing verification: corrupted headers = wrong source ID = missed bit = test failure

4. Routing Variants — Design Changes

4.1 HOP-Based (Baseline)

The baseline router uses unary-encoded relative hop counts embedded in the packet header. Each router decrements the hop counter before forwarding.

- hopX unary encoding: 0 hops=4'h0, 1 hop=4'h1, 2 hops=4'h3, 3 hops=4'h7
- Routing decision: if hopX[0]=1 → route X (direction from dirX bit); else if hopY[0]=1 → route Y; else → PE
- Header modification: output_ctrl right-shifts hopX and hopY at every router: hopX >>= 1
- Limitation: every router must read-modify-write the header — switching activity on 8 bits per hop

4.2 XY-Standard (XYSTD) — Changes from HOP

XYSTD replaces the mutable hop counter with static absolute destination coordinates. The packet header is never modified in transit.

File Changed	HOP Behavior	XYSTD Change
input_ctrl_xystd.v	Check hopX[0] + dirX bit	Compare dst_x to ROUTER_X parameter (8-bit comparator)
input_ctrl_xystd.v	Extract pkt[55:52] as hopX	Extract pkt[55:48] as dst_x, pkt[47:40] as dst_y
input_ctrl_xystd.v	No ROUTER_X/Y needed	Add parameter ROUTER_X/Y — router needs own coordinates
output_ctrl_xystd.v	Decrement: hopX >>= 1, hopY >>= 1	Delete both shift lines — packet passed through unchanged
gold_router_xystd.v	No coordinate parameters	Add ROUTER_X/Y params; pass to all 5 input_ctrl instances
gold_mesh_xystd.v	No coord instantiation	Pass hardcoded ROUTER_X/Y to each of 16 router instances

4.3 Torus — Changes from XYSTD

Torus routing adds wrap-around links in both dimensions and replaces the comparator with a 2-bit modular subtractor. Only 2 files require logic changes; 2 more require only wiring changes.

Modular Distance Computation

The key insight is that 2-bit binary subtraction naturally computes (a − b) mod 4 — no explicit modulo hardware needed:

dx_mod value	Meaning	Direction	Mesh hops	Torus hops
2'b00	dst_x == ROUTER_X	X done → check Y	0	0
2'b01	1 hop East	Go East	1	1

dx_mod value	Meaning	Direction	Mesh hops	Torus hops
2'b10	2 hops (tie)	Go East (convention)	2	2
2'b11	3 hops East = 1 hop West wrap	Go West via wrap	3	1 ✓

File-Level Changes for Torus

File	Change Type	Exact Change
input_ctrl_torus.v	Logic change	Replace comparator with: wire [1:0] dx_mod = dst_x[1:0] - ROUTER_X[1:0]
input_ctrl_torus.v	Logic change	Routing: if dx_mod==2'b11 → West wrap; else East (dx_mod 01 or 10)
gold_router_torus.v	Name change only	Instantiate input_ctrl_torus; reuse output_ctrl_xystd unchanged
gold_mesh_torus.v	Wiring change	Add 48 wrap wires (16 groups × 3 signals: so/ro/do)
gold_mesh_torus.v	Wiring change	Replace si=1'b0 tie-offs with live wrap connections on all 8 boundary routers
output_ctrl_xystd.v	Unchanged	Reused verbatim — output controller is topology-agnostic

Wrap Link Connections (gold_mesh_torus.v)

Wrap Type	From (XYSTD: tied off)	To (Torus: live connection)
Horizontal (×4)	col-3 East output tied to 1'b0	col-3 East output → col-0 West input (all 4 rows)
Horizontal (×4)	col-0 West output tied to 1'b0	col-0 West output → col-3 East input (all 4 rows)
Vertical (×4)	row-3 North output tied to 1'b0	row-3 North output → row-0 South input (all 4 cols)
Vertical (×4)	row-0 South output tied to 1'b0	row-0 South output → row-3 North input (all 4 cols)

5. Synthesis Results (DC Compiler, FreePDK45 45nm)

All three router variants were synthesized using Synopsys Design Compiler K-2015.06-SP5-5 targeting the NCSU FreePDK45 45nm standard cell library at 250 MHz. Timing was met for all three variants.

5.1 Timing Results

Timing Metric	HOP (Baseline)	XYSTD	TORUS
Target Clock Period (ns)	4.000	4.000	4.000
Critical Path (ns)	**3.820**	3.790	3.940
Timing Slack (ns)	**+0.130 ✓**	+0.150 ✓	+0.010 ✓
Max Achievable Frequency (MHz)	**258.4**	259.7	250.6
vs HOP Baseline	**—**	+0.5% ✓	−3.0%

The torus critical path is 150ps longer than XYSTD due to the wrap-around mux added to the input_ctrl routing decode. The 2-bit subtractor plus wrap-direction mux adds one additional logic level compared to the plain comparator in XYSTD. Despite this, timing closure is achieved at 250 MHz with 10ps of slack.

5.2 Area Results

Area Metric	HOP (Baseline)	XYSTD	TORUS
Total Cell Area (µm²)	30,135.2	**30,100.0**	29,974.7
Gate Equivalents (GE)	21,463.8	**21,438.7**	21,349.1
vs HOP Baseline	—	**−0.1% ✓**	−0.5% ✓
Full 16-Router Mesh (µm²)	482,163	**481,600**	479,595

All three variants have nearly identical area because the routing decode logic represents only ~2% of total router area. The dominant components — 20 VC buffer registers (1,280 bits of flip-flops) and 5 crossbar multiplexers (5 × 64-bit 4-to-1 muxes) — are identical across all variants. The torus slightly undercuts both variants because the 2-bit modular subtractor is smaller than the 4-bit unary shift decoder used in HOP.

5.3 Power Results

Power Metric	HOP (Baseline)	XYSTD	TORUS
Dynamic Power (mW)	**5.769**	5.499	5.726
Leakage Power (µW)	133.92	**133.94**	133.63
Total Power (mW)	**5.903**	5.633	5.859

Power Metric	HOP (Baseline)	XYSTD	TORUS
vs HOP Baseline	—	−4.6% ✓	−0.7% ✓
Full Mesh ×16 (mW)	**94.45**	90.12	93.74

XYSTD achieves the best power because static headers eliminate switching activity on 8 bits (hopX + hopY fields) at every router hop. In HOP, those 8 flip-flops toggle at every router crossing. In XYSTD, the header never changes, so those bits never toggle. The power reduction is concentrated in dynamic power (−4.7%) while leakage is essentially unchanged across all variants.

6. Performance Analysis — Latency Comparison

Simulation was conducted at 500 MHz (CLK_HALF = 1ns) across 36 representative pairs covering all hop-distance categories from 1 to 6 hops. All 36/36 pairs were successfully measured for each variant.

6.1 Latency Summary

Latency Metric	HOP	XYSTD	TORUS
Pairs Measured	36/36	36/36	36/36
Min Latency (cycles)	4	4	4
Max Latency (cycles)	14	**14**	10
Avg Latency (cycles)	7.22	**7.22**	5.44
Min Latency (ns @ 500MHz)	8.0	8.0	8.0
Max Latency (ns @ 500MHz)	28.0	**28.0**	20.0
Avg Latency (ns @ 500MHz)	14.4	**14.4**	10.9
Avg Latency (real silicon, ns)	27.9	**27.8**	21.7
Max Latency (real silicon, ns)	54.2	**53.9**	39.9

6.2 Key Latency Findings

HOP vs XYSTD — Zero Latency Difference

HOP and XYSTD produce identical latency for every single pair tested. This is expected: changing the routing decode from a bit-shift to a comparator does not alter the number of hops a packet traverses. Both variants use XY dimension-order routing on the same mesh topology. The only difference is how the routing decision is computed — the path taken is identical.

Torus — 24.6% Average Latency Reduction

- Average latency: 5.44 cycles vs 7.22 cycles on mesh — a 24.6% improvement
- Maximum latency: 10 cycles vs 14 cycles — a 28.6% reduction (eliminates the worst-case tail)
- Minimum latency: unchanged at 4 cycles — short-distance pairs see no benefit

Corner-to-Corner Example: (0,0) → (3,3)

Topology	Route Taken	Hops	Cycles
Mesh (HOP/XYSTD)	3 East + 3 North = 6 hops	6	14
Torus	1 West wrap + 1 South wrap = 2 hops	2	6 ✓

6.3 Selected Pair Latency Detail

Pair	HOP	XYSTD	TORUS	Δ (cyc)	Torus route	Mesh route	Winner
(0,0)→(1,0)	4	4	4	0	1E	1E	Tie
(0,0)→(3,0)	8	8	4	−4	1W wrap	3E	Torus ✓
(0,0)→(0,3)	8	8	4	−4	1S wrap	3N	Torus ✓
(0,0)→(3,3)	14	14	6	−8	1W+1S wrap	3E+3N	Torus ✓
(3,0)→(0,3)	14	14	6	−8	1E+1S wrap	3W+3N	Torus ✓
(0,0)→(2,2)	10	10	10	0	2E+2N	2E+2N	Tie
(0,0)→(2,0)	6	6	6	0	2E	2E	Tie
(0,0)→(3,2)	12	12	8	−4	1W+2N wrap	3E+2N	Torus ✓

Note: Pairs where dx_mod or dy_mod = 2'b10 (2 hops, tie case) see no torus improvement — the wrap path is the same distance as the forward path. Torus wins only when one dimension has mod=3 (3 forward = 1 wrap).

7. Consolidated PPA Comparison

7.1 Normalised PPA (HOP = 1.000×)

Metric (normalised)	HOP	XYSTD	TORUS
Area	1.000×	**0.999× ✓**	0.995× ✓
Total Power	**1.000×**	0.954× ✓	0.993× ✓
Max Frequency	**1.000×**	1.005× ✓	0.970×
Avg Latency	1.000×	**1.000×**	0.754× ✓
Max Latency	1.000×	**1.000×**	0.714× ✓

7.2 PPA Trade-off Analysis

For Fixed Mesh Deployments — XYSTD Dominates HOP

- XYSTD is strictly better than HOP on every metric: area, power, frequency, and latency
- Static headers eliminate the need to read-modify-write 8 header bits at every router hop
- Power saving of 4.6% per router = 4.33 mW savings across the full 16-router mesh
- There is no reason to use HOP routing in any new mesh implementation

The Torus Trade-off

- Torus pays 3.0% frequency penalty (250.6 vs 258.4 MHz) for the wrap-around mux logic
- Torus gains 24.6% average latency reduction and 28.6% maximum latency reduction
- Energy per packet: lower despite slightly higher per-cycle power because packets spend 25% fewer cycles in transit
- Area cost: negligible at 0.5% reduction (torus routing decode is actually smaller than HOP's shift decoder)
- Wiring cost: physical wrap links must connect non-adjacent grid edges — manageable for 4×4, complex for larger grids

Recommendation

Deployment Scenario	Recommended Variant
Fixed 4×4 mesh, power-constrained	XYSTD — best power, identical latency to HOP, zero extra wiring
4×4 with all-to-all or long-distance traffic	TORUS — 25% latency gain with negligible PPA cost
Any scenario vs HOP baseline	XYSTD or TORUS — HOP is strictly dominated by both

8. Cardinal NIC

8.1 NIC Design Overview

The Cardinal NIC provides the interface between a processing element (PE) and the NoC router. It abstracts the router handshake protocol from the processor, managing packet construction, injection timing, and reception buffering.

8.2 NIC Functionality

- Store operation: processor writes data → NIC constructs packet header → injects to router when polarity=1 and peri=1
- Load operation: NIC monitors router peso signal → captures arriving packet → presents to processor
- Buffer management: tracks when NIC input/output buffers are available or unavailable
- Handshake bridging: translates processor bus transactions to router ready/send protocol

8.3 NIC Test Coverage

Test Case	Condition	Expected Behavior
Reset verification	NIC reset asserted	All buffers clear, outputs deasserted
Store — buffer available	NIC output buffer empty	Packet constructed and injected in 2 cycles
Store — buffer unavailable	NIC output buffer full	Processor stalled; injection delayed until buffer clears
Load — buffer available	Packet arriving from router	Data captured and presented to processor
Load — buffer unavailable	NIC input buffer full	Router pero deasserted; backpressure applied upstream
Handshake without blocking	pero=1 throughout	Full pipeline throughput, 2-cycle NIC latency
Handshake with blocking	pero toggled mid-transfer	Packet held; flushed on reassert; no data loss

9. Conclusions

9.1 Summary of Achievements

- All three routing variants implemented in fully synthesizable RTL Verilog
- 328/328 test assertions pass across all TC1–TC14 test cases
- Synthesis timing closure achieved for all three variants at 250 MHz target
- Torus demonstrates 24.6% average latency improvement over mesh at negligible PPA cost
- XYSTD demonstrates 4.6% power improvement over HOP with zero latency impact
- Full gather testbench generates all 16 gather_phasek.res output files with correct format

9.2 Key Insights

The 2% Rule

Routing decode logic — the component that changes between all three variants — constitutes only ~2% of total router area and power. The remaining 98% is dominated by the 20 VC buffer registers and 5 crossbar multiplexers, which are identical across all variants. This is why the PPA differences between variants are small even though the routing algorithms are fundamentally different.

Topology vs Algorithm

The torus demonstrates that topology (wrap-around links) provides dramatically more latency benefit than algorithm (how routing decisions are computed). XYSTD vs HOP is an algorithm change with zero latency impact. Torus is a topology change with 25% latency impact. For NoC design, topology choice is the dominant latency knob.

Virtual Channel Sufficiency

Two virtual channels per port is sufficient for deadlock freedom with XY routing in both mesh and torus topologies. Increasing to 4 VCs would double buffer area (from 1,280 to 2,560 flip-flops) and double leakage power with no latency benefit for the workloads tested. Two VCs is the optimal point for this design.

9.3 Final PPA Scorecard

Category	HOP	XYSTD	TORUS
Timing (freq)	**258.4 MHz**	259.7 MHz ★	250.6 MHz
Area	30,135 µm²	**30,100 µm²**	29,975 µm² ★
Power	**5.903 mW**	5.633 mW ★	5.859 mW
Avg Latency	7.22 cyc	**7.22 cyc**	5.44 cyc ★
Max Latency	14 cyc	**14 cyc**	10 cyc ★
Category Wins	0	**2**	3

Category	HOP	XYSTD	TORUS
Overall Verdict	Baseline	**Best for mesh**	Best overall ★

End of Report

USC Viterbi EE577B · Spring 2026 · Om Bhatt & Melvin Houston